Smart Campus Incident and Maintenance Prioritization

Python Programming – Activity-Based Learning (ABL) Assignment

1. Problem Statement

Classify hypothetical maintenance reports using severity, location, and reported frequency; rank cases using predefined rules and generate a priority summary.

2. Objectives

• Store maintenance incidents. • Calculate a priority score from severity, location, and frequency. • Classify each report as Critical, High, Medium, or Low. • Rank reports and summarize priority counts.

3. Assumptions and Priority Policy

Severity uses a 1–5 scale, where 5 is safety-critical. Frequency uses a 1–5 scale, where 5 means repeatedly reported. The sample score is severity × 3 + location weight + frequency × 2. Severity 5 or score at least 18 is Critical; score 13–17 is High; score 8–12 is Medium; otherwise Low. Location weights are illustrative and can be adjusted to campus policy. These reports are hypothetical.

4. Algorithm

```
1. Store incident ID, description, severity, location, and frequency.
2. Look up the location weight.
3. Calculate score = severity*3 + location weight + frequency*2.
4. Assign priority using the predefined thresholds.
5. Sort by score descending, then severity and frequency.
6. Print the ranked incident list and counts by priority.
```

5. Flowchart (Text Representation)

```
START
  |
Read incident reports
  |
For each report: get location weight
  |
Calculate priority score
  |
Classify Critical / High / Medium / Low
  |
Sort reports by priority score
  |
Display ranking and priority summary
  |
 END
```

6. Python Source Code

```
# Smart Campus Incident and Maintenance Prioritization
# Rank hypothetical maintenance reports using severity, location, and frequency.

reports = [
    {"id": "INC101", "title": "Electrical sparking", "severity": 5, "location": "Main Lab", "frequency": 3},
    {"id": "INC102", "title": "Water leakage", "severity": 4, "location": "Hostel Block", "frequency": 5},
    {"id": "INC103", "title": "Broken classroom fan", "severity": 2, "location": "Classroom", "frequency": 4},
    {"id": "INC104", "title": "Blocked emergency exit", "severity": 5, "location": "Library", "frequency": 2},
    {"id": "INC105", "title": "Damaged corridor light", "severity": 3, "location": "Corridor", "frequency": 3},
    {"id": "INC106", "title": "Washroom tap repair", "severity": 2, "location": "Washroom", "frequency": 2},
]

# Example policy: severity (1-5) is weighted most; location and frequency add points.
# Higher scores mean higher priority. Safety-critical severity 5 is marked Critical.
LOCATION_WEIGHT = {
    "Main Lab": 3,
    "Hostel Block": 3,
    "Classroom": 2,
    "Library": 2,
    "Corridor": 2,
    "Washroom": 1,
}

def classify_priority(severity, score):
    if severity == 5 or score >= 18:
        return "CRITICAL"
    elif score >= 13:
        return "HIGH"
    elif score >= 8:
        return "MEDIUM"
    return "LOW"

for report in reports:
    location_points = LOCATION_WEIGHT.get(report["location"], 1)
```

```
    report["score"] = report["severity"] * 3 + location_points + report["frequency"] * 2
    report["priority"] = classify_priority(report["severity"], report["score"])

ranked_reports = sorted(
    reports,
    key=lambda r: (-r["score"], -r["severity"], -r["frequency"], r["id"])
)

print("SMART CAMPUS INCIDENT & MAINTENANCE PRIORITIZATION")
print("=" * 96)
print("Policy: score = severity*3 + location weight + frequency*2")
print("Severity scale: 1 (minor) to 5 (safety-critical); frequency scale: 1 (rare) to 5 (repeated)")
print("-" * 96)
print(f"{'Rank':<6}{'Incident':<10}{'Report':<27}{'Severity':>10}{'Location':<16}{'Freq.':>7}{'Score':>8}{'Priority':>12}")
print("-" * 96)

for rank, report in enumerate(ranked_reports, start=1):
    print(f"{rank:<6}{report['id']:<10}{report['title']:<27}{report['severity']:>10}"
          f"{report['location']:<16}{report['frequency']:>7}{report['score']:>8}{report['priority']:>12}")

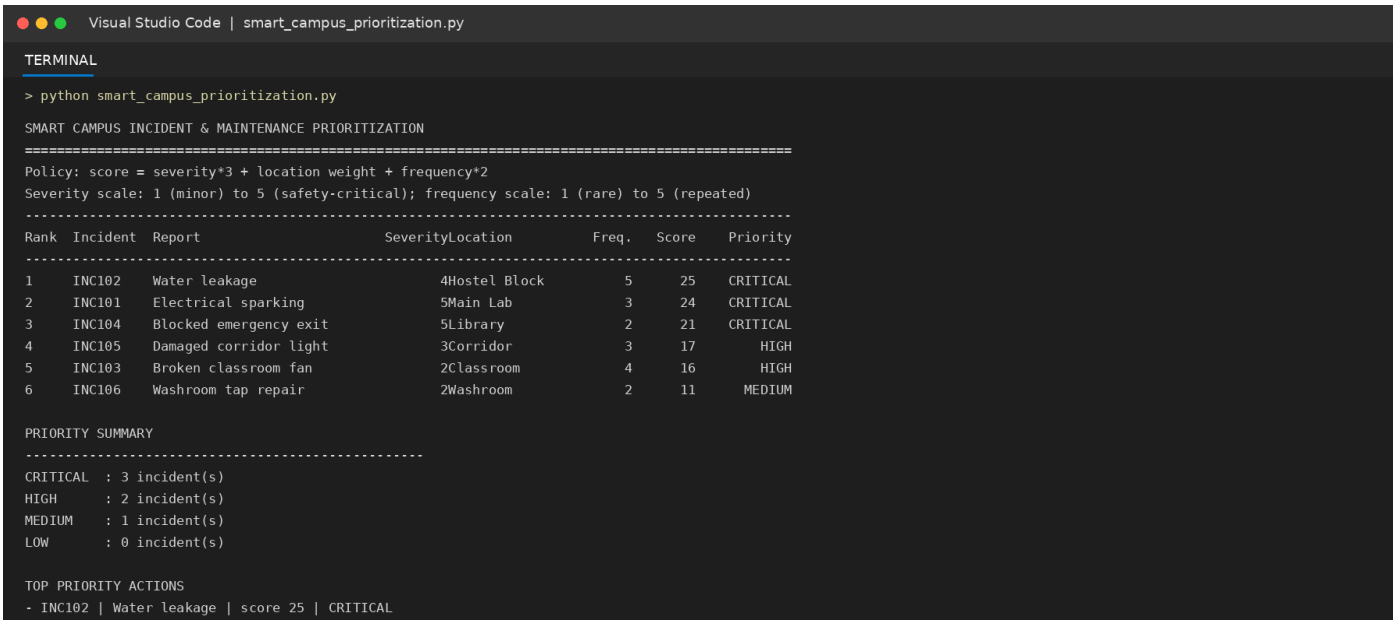
print("\nPRIORITY SUMMARY")
print("-" * 50)
for level in ("CRITICAL", "HIGH", "MEDIUM", "LOW"):
    count = sum(1 for r in reports if r["priority"] == level)
    print(f"{level:<10}: {count} incident(s)")

print("\nTOP PRIORITY ACTIONS")
for report in ranked_reports[:3]:
    print(f"- {report['id']} | {report['title']} | score {report['score']} | {report['priority']}")

print("\nResult: Incident reports ranked and priority summary generated.")
print("Note: These are hypothetical records and example prioritization rules.")
```

7. Sample Execution Output Screenshot

This terminal-style screenshot shows sample output. Run smart_campus_prioritization.py in Visual Studio Code to reproduce it.



8. Test Cases

1. Severity 5 report -> classified Critical under the example policy.
2. Larger reported frequency -> adds 2 points per frequency level.
3. Main Lab / Hostel Block -> location weight 3 in this sample.
4. Reports are ranked by score descending.
5. Equal scores are resolved by severity, frequency, then incident ID.
6. Priority summary counts incidents in each category.

9. Conclusion

The program calculates scores from severity, location, and frequency, classifies and ranks hypothetical maintenance reports, and prints a summary to help staff identify urgent cases. Real campus safety procedures should override any illustrative score.